

Xây dựng Trợ lý Dữ liệu Tài chính AI (AI Financial Data Assistant)

Day 2: Advanced Methods, Production-Ready Requirements

Presenter: Nguyễn Tấn Minh, Đỗ Hoàng Hà

Japan Advanced Institute of Science and Technology
VNU University of Engineering and Technology

August, 21 2026

Table of contents

Tổng quan bài toán End-to-End Financial QA
Thách thức Cấu trúc & Bảng Phân cấp (MultiHiertt)
Lập trình Hỗ trợ Suy luận (PAL & Binder)
Triệt tiêu Ảo giác Số học (Numeric Masking)
Suy luận Bảng Động & Scalable Table-RAG
Hệ thống Multi-Agent & An toàn Thực thi
Kiến trúc Tổng thể
Serving LLM theo Hạ tầng
Chọn LLM theo Vai trò Agent
Host Đa Model trên 1 GPU
Latency Budget cho Pipeline Multi-Agent
Sandbox Thực thi An toàn
Tài liệu Tham khảo (References)

Tổng quan bài toán End-to-End Financial QA

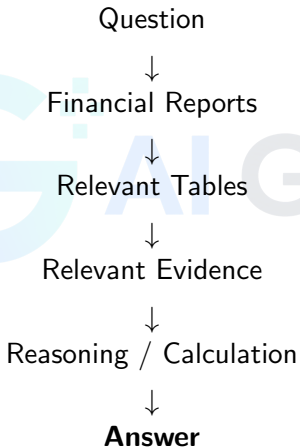
Bài toán Financial Question Answering

- ▶ Đầu vào là một **câu hỏi tài chính bằng ngôn ngữ tự nhiên**.
- ▶ Nguồn dữ liệu là một tập lớn các **báo cáo tài chính**.
- ▶ Hệ thống cần:
 1. Tìm đúng báo cáo.
 2. Tìm đúng bảng.
 3. Tìm đúng dữ liệu cần thiết.
 4. Thực hiện tính toán để trả lời câu hỏi.

Ví dụ

Doanh thu của công ty X năm 2023 tăng bao nhiêu phần trăm so với năm 2022?

End-to-End Financial QA Pipeline



2.1 Giới hạn của Text-to-Pandas với Bảng Biểu Tài chính

Giả định ngây thơ (Flat Table)

- ▶ Bảng là ma trận 2D chuẩn $M \times N$.
- ▶ Mỗi cột có đúng 1 header duy nhất.
- ▶ Mỗi dòng độc lập, không có quan hệ cha-con.
- ▶ Text và Table tách rời nhau.

Thực tế trong Báo cáo Tài chính (10-K/10-Q)

- ▶ **Multi-level Hierarchical Headers:** Cột/Dòng lồng nhau 3-4 cấp.
- ▶ **Merged Cells / Spans:** Ô gộp, dòng tổng con (*Subtotals*).
- ▶ **Cross-table Linking:** Câu hỏi yêu cầu lấy số từ 2 bảng khác nhau + Chú thích chân trang (Footnotes).

2.2 MultiHieertt Benchmark [1]

Đặc điểm dữ liệu:

- ▶ Trích xuất từ **Báo cáo thường niên (Annual Reports)** thực tế của các công ty niêm yết.
- ▶ Mỗi mẫu gồm **nhiều bảng phân cấp phức tạp** kèm các đoạn văn bản giải trình.
- ▶ Yêu cầu suy luận toán học
Multi-hop: Tìm ô ở Bảng 1, đối chiếu với điều kiện ở Bảng 2 và tỷ lệ ở Text.

Citation: Zhao et al. [1], ACL 2022.

Thách thức chính:

- ▶ **Hierarchical Path Matching**: Phải lần theo đường dẫn phân cấp.
- ▶ **Cross-Evidence Aggregation**: 70%+ câu hỏi yêu cầu kết hợp ≥ 3 sự kiện rải rác.
- ▶ Độ chính xác của các mô hình LLM cơ bản (GPT-3/BERT) giảm sút nghiêm trọng.

2.3 Giải pháp: Hierarchical Schema Encoding & Tree Traversal

Làm thế nào để đưa bảng phân cấp vào LLM mà không làm mất thông tin?

1. Coordinate & Header Path Linearization [2]

Mỗi ô dữ liệu được gán nhãn với toàn bộ cây phân cấp của nó:

$$c_{i,j} = (\text{RowPath}(i), \text{ColPath}(j), \text{Value})$$

Ví dụ: [Year: 2023 | Segment: Cloud | Metric: Gross Margin] = 68.4%

2. Graph / Tree Representation [1]

- ▶ Biểu diễn bảng dưới dạng cây hoặc đồ thị không đồng nhất (*Heterogeneous Graph*).
- ▶ Các cạnh nối quan hệ *Parent-Child*, *Sibling*, và *Semantic Cross-reference* giữa Text và Cell.
- ▶ Cho phép mô hình duyệt cây chính xác trước khi sinh mã.

2.4 Kết quả Thực nghiệm trên MultiHiertt Benchmark

Mô hình (Model)	Dev		Test	
	EM	F ₁	EM	F ₁
Longformer + Reasoning	2.71	6.93	2.86	6.23
Facts Retrieving + TAPAS [3]	8.94	10.70	7.67	10.04
Facts Retrieving + NumNet	10.32	12.59	10.77	12.02
TAGOP (RoBERTa-large) [4]	19.16	21.08	17.81	19.35
Facts Retrieving + Seq2Prog	26.19	28.74	24.58	26.30
FinQANet (RoBERTa-large) [5]	32.41	35.37	31.72	33.60
MT2Net (BERT-base) [1]	33.68	35.94	32.07	33.67
MT2Net (BERT-large)	34.03	36.13	33.25	34.98
MT2Net (RoBERTa-base)	35.69	37.81	34.32	36.17
MT2Net (RoBERTa-large)	37.05	39.96	36.22	38.43
Human Expert	–	–	83.12	87.03

Table: Kết quả so sánh trên MultiHiertt [1].

Phân tích cốt lõi (Key Takeaways)

- ▶ **Khoảng cách lớn với con người:**
Chuyên gia đạt **83.12% EM** vs. Model SOTA chỉ **36.22% EM** (chênh lệch > 46%).
- ▶ **Encoder truyền thông thất bại:**
Longformer (2.86%) và TAPAS [3] (7.67%) bất lực trước bài toán đa bảng.
- ▶ **Seq2Prog / Program là chìa khóa:**
Sinh chương trình (FinQANet [5], MT2Net) tăng vọt từ ~ 10% lên > 36%.

3.1 Tại sao Chain-of-Thought (CoT) thuần Text thất bại?

Vấn đề của CoT

1. **Tokenization Fragmentation:** Số thực (e.g. 142.857) bị cắt thành các subword token ngẫu nhiên (14, 2, 857), làm mất cảm nhận độ lớn.
2. **Hallucinated Arithmetic:** LLM dự đoán token tiếp theo theo xác suất, không thực hiện phép tính số học thực sự.
3. **Lỗi tích lũy (Error Cascading):** Trong bài toán 4-5 bước tính, sai 1 bước dẫn đến toàn bộ kết quả vô giá trị.

Nguyên lý cốt lõi của FinNLP hiện đại

Phân rã trách nhiệm (Decoupling):

- **LLM:** Đóng vai trò **Planner** (Hiểu ngôn ngữ tự nhiên, lập luận logic, sinh mã).
- **Python / SymPy Interpreter:** Đóng vai trò **Calculator / Executor** (Thực thi số học chính xác 100%).

Citation: Program of Thoughts (PoT) [6], TMLR / ICLR 2023.

3.2 PAL: Program-aided Language Models [7]

Thay vì sinh lời giải bằng văn bản, LLM sinh các bước tư duy trực tiếp dưới dạng **mã nguồn Python kèm comment giải thích**.

```
# Question: In 2023, what was the YoY growth
    rate of Operating Income?
# Evidence: 2022 OpInc = $4,500M, 2023 OpInc =
    $5,310M

# Step 1: Define extracted financial variables
op_inc_2022 = 4500.0
op_inc_2023 = 5310.0

# Step 2: Compute YoY growth rate
growth_rate = (op_inc_2023 - op_inc_2022) /
    op_inc_2022

# Step 3: Format answer as percentage
ans = growth_rate * 100
```

Ưu điểm vượt trội:

- ▶ Xử lý hoàn hảo các phép toán phức tạp: lũy kế, lãi kép, ma trận tỷ suất.
- ▶ Dễ dàng debug và kiểm tra logic trung gian.
- ▶ Tăng **+15% đến +25%** độ chính xác so với CoT trên FinQA [5] và GSM8K.

3.3 Binder: Binding Language Models in Symbolic Languages [8]

Mở rộng ngôn ngữ biểu tượng (SQL/Python) bằng cách cho phép nhúng trực tiếp các hàm gọi LLM (*LLM API primitives*) vào bên trong chương trình.

```
-- Binder SQL Example for Hybrid Reasoning
SELECT
  company_name,
  revenue_2023,
  -- Call LLM as a semantic function inside SQL
  _qa(concat("Is ", segment, " part of Cloud AI?"
    )) AS is_cloud_ai
FROM financial_table
WHERE year = 2023 AND is_cloud_ai = 'Yes'
ORDER BY revenue_2023 DESC
LIMIT 1;
```

Đột phá của Binder:

- ▶ Giải quyết bài toán: SQL thuần túy không hiểu ngữ nghĩa mờ (*fuzzy semantics*), còn LLM thuần túy không giỏi tổng hợp dữ liệu lớn.
- ▶ Tận dụng SQL cho **Filter/Sort/Aggregate** và LLM cho **Semantic Mapping**.

Citation: Cheng et al. [8], ICLR 2023.

4.1 Cơ chế Numeric Masking / De-lexicalization

Vấn đề: Khi sinh code, LLM hay "nhớ vệt" số trong prompt hoặc tự bịa ra hằng số ngẫu nhiên do bias từ pre-training.

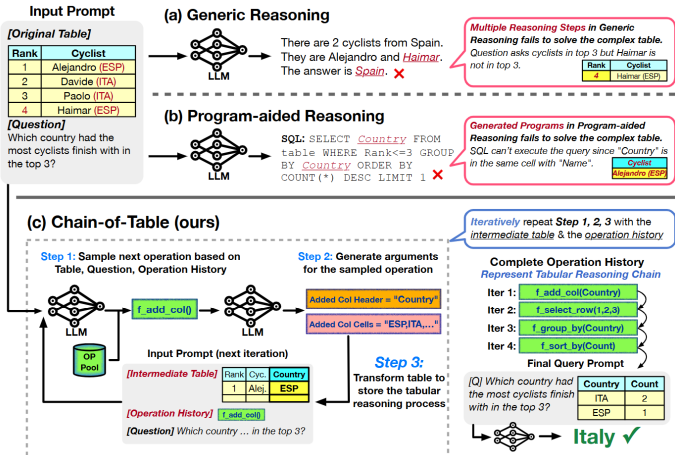
Quy trình 3 bước De-lexicalization

1. **Masking Context & Query:** Thay thế toàn bộ số thực bằng các placeholder tượng trưng:
"Tăng từ \$500M lên \$650M" "Tăng từ [NUM_0] lên [NUM_1]"
2. **Symbolic Program Generation:** LLM chỉ sinh ra cấu trúc biểu thức: $ans = ([NUM_1] - [NUM_0]) / [NUM_0]$
3. **Deterministic Value Binding:** Map ngược lại giá trị gốc vào execution runtime.

Lợi ích trong Production

- ▶ **100% Grounding:** Không một con số nào có thể bị ảo giác (hallucinated), vì code chỉ chứa biến trở về đúng bằng.
- ▶ **Generalization across Entities:** Cùng 1 logic tính ROE/EBITDA áp dụng được cho mọi công ty mà không bị ảnh hưởng bởi quy mô doanh thu.
- ▶ **Giảm chiều không gian Token:** Giảm đáng kể hiện tượng out-of-vocabulary hoặc split token kỳ quặc.

5.1 Chain-of-Table (ICLR 2024 - Google DeepMind) [9]



5.2 TableRAG: Million-Token Table Understanding [10]

Nút thắt của RAG truyền thống

- ▶ Các bảng lớn trong báo cáo tài chính kiểm toán có thể lên tới hàng nghìn dòng.
- ▶ Cắt nhỏ (chunking) theo dòng làm **mất tiêu đề cột** và **mất tính liên kết toàn cục**.
- ▶ Đưa toàn bộ bảng vào Prompt gây vượt context limit và tốn chi phí (Token Inflation).

Giải pháp TableRAG [10]

Cơ chế truy xuất 2 tầng:

1. **Schema Retrieval:** Trích xuất các cột/hàng liên quan dựa trên metadata và schema index.
2. **Cell-targeted Pointers:** Chỉ lấy các ô dữ liệu giao điểm (*Intersection Cells*) phục vụ trực tiếp cho câu hỏi.
3. Xử lý bảng hàng triệu token với độ trễ dưới 1 giây.

6.1 Multi-Agent Collaboration & Reflection [11]

Phân vai chuyên biệt (Specialized Roles)

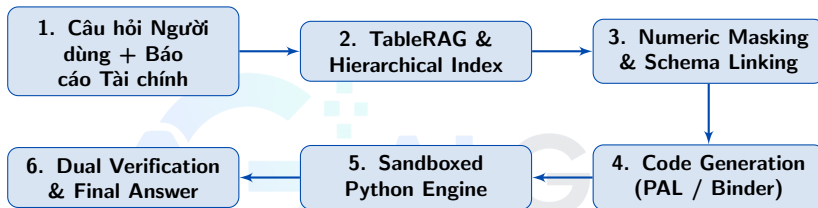
- ▶ **Planner Agent:** Phân rã câu hỏi đa bước.
- ▶ **Retriever Agent:** Định vị bảng, footnote và cell.
- ▶ **Programmer Agent:** Viết mã Python/Pandas.
- ▶ **Critic / Verifier Agent:** Kiểm tra logic kế toán, đối chiếu số liệu và phát hiện ngoại lệ.

Citation: CLER [11], EMNLP 2024.

Vòng lặp Tự chỉnh sửa (Reflection Loop) [11]

- ▶ Khi Sandbox báo lỗi (KeyError, ZeroDivision):
Traceback → Feedback → Self-Correction
- ▶ **Dual Verification:** So khớp kết quả thực thi của Python với văn bản giải thích. Nếu lệch nhau → kích hoạt lượt sinh lại.

7.1 Kiến trúc Trợ lý Dữ liệu Tài chính AI Hoàn chỉnh



Đặc điểm kiến trúc

Kết hợp hoàn chỉnh: **Truy xuất chính xác (TableRAG [10]) → Khử ảo giác (Masking [5]) → Lập luận biểu tượng (PAL [7]) → Thực thi an toàn (Sandbox) → Kiểm định kép (Verification).**

Tổng kết

3 Bài học Cốt lõi cho AI Tài chính:

1. **Cấu trúc là trên hết:** Đừng làm phẳng (flatten) bảng biểu một cách ngẫu thơ. Hãy bảo toàn cây phân cấp và metadata của từng ô.
2. **Tách biệt Suy luận và Tính toán:** Luôn dùng LLM để lập trình (Program-of-Thought) và giao việc tính toán cho Sandbox Python.
3. **Triệt tiêu ảo giác bằng Masking & Verification:** Che dấu số thực khi sinh code và thiết lập vòng lặp kiểm tra chéo đa tầng.

Serving LLM theo Hạ tầng

Vì sao Cần Serving Engine, Không Chỉ Load Model

Load thẳng model qua transformers/PyTorch xử lý **tuần tự từng request** - GPU phần lớn thời gian rảnh trong lúc chờ decode từng token, không tận dụng được song song.

Insight cốt lõi: Continuous Batching + PagedAttention

Gộp động nhiều request đang decode dở dang vào cùng 1 batch tại mỗi bước sinh token (thay vì chờ cả batch tính xong mới nhận request mới); quản lý KV-cache theo trang bộ nhớ (paging) như hệ điều hành quản lý RAM → giảm phân mảnh, tăng throughput **gấp nhiều lần**.

Kwon et al., 2023 (SOSP) - *"Efficient Memory Management for Large Language Model Serving with PagedAttention"* - arxiv.org/abs/2309.06180

Năm Lựa chọn Serving Engine theo Hạ tầng

Công cụ	Ưu thế	Phù hợp hạ tầng
vLLM	Throughput cao, PagedAttention, cộng đồng lớn, dễ vận hành	Cụm nhiều GPU, host chung nhiều model
TensorRT-LLM	Tối ưu sâu nhất cho NVIDIA, latency thấp nhất	GPU NVIDIA cố định, ít đổi kiến trúc model
SGLang	RadixAttention - tái dùng prefix lặp lại giữa các lời gọi	Multi-agent, prompt template lặp lại nhiều lần
llama.cpp / Ollama	Quantize sâu, chạy được cả CPU	Máy đơn, không GPU rời, prototype
LM Studio	Hỗ trợ MLX - tối ưu riêng cho Apple Silicon (unified memory)	Máy Mac (M-series), phát triển/thử nghiệm local

Zheng et al., 2024 (NeurIPS) - "*SGLang: Efficient Execution of Structured Language Model Programs*" - arxiv.org/abs/2312.07104

Chọn LLM theo Vai trò Agent

Không Phải Agent Nào Cũng Cần Model Lớn Nhất

Kiến trúc Multi-Agent (Planner / Retriever / Programmer / Critic) không đòi hỏi **1 model to dùng chung** - mỗi vai trò có một dạng năng lực khác nhau, có thể (và nên) gán model khác nhau cho từng vai trò.

Vai trò	Yêu cầu năng lực chính	Gợi ý loại model
Planner	Suy luận đa bước, phân rã câu hỏi	Model reasoning mạnh nhất hệ thống
Retriever	Ra quyết định định vị bảng/cell, diễn giải kết quả truy xuất	Model vừa/nhỏ
Programmer	Sinh code Python/Pandas chính xác	Model chuyên code-generation
Critic/Verifier	Đôi chiếu logic, phát hiện sai lệch	Model nhỏ, chạy nhanh - gọi lặp trong reflection loop

Đánh đổi Năng lực - Chi phí - Độ trễ theo Vai trò

Nguyên tắc phân bổ ngân sách tính toán không đồng đều giữa các vai trò:

Chạy 1 lần, quyết định cả hướng đi → đáng dùng model đắt

Planner sai ngay từ đầu kéo sai toàn bộ chuỗi phía sau - lỗi ở đây không sửa được bằng các bước sau.

Lặp lại nhiều lần trong 1 câu hỏi → nên dùng model rẻ/nhanh

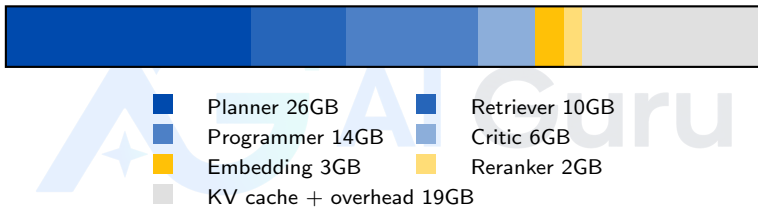
Critic có thể chạy lại nhiều vòng trong reflection loop - dùng model lớn ở đây nhân chi phí lên trực tiếp theo số vòng lặp.

Wang et al., 2024 - *"Mixture-of-Agents Enhances Large Language Model Capabilities"* - arxiv.org/abs/2406.04692

Host Đa Model trên 1 GPU

Ngân sách VRAM khi Host Nhiều Model Cùng Lúc

6 loại model (4 LLM agent + embedding + reranker) cùng cạnh tranh chung một ngân sách VRAM giới hạn (vd. $1 \times \text{GPU } 80\text{GB}$):



Vấn đề thường bị bỏ qua

Embedding & reranker chạy **liên tục ở mọi câu hỏi** - nếu không dành riêng ngân sách ổn định, bị LLM lớn chiếm hết VRAM vào giờ cao điểm.

Quantization để Vừa Nhiều Model

Nén trọng số model (AWQ/GPTQ, INT4/INT8) giảm kích thước **2–4 lần**, đổi lấy một phần độ chính xác nhỏ để có chỗ host thêm model khác trên cùng GPU.

Nguyên tắc phân bổ cho LLM sinh văn bản

Quantize sâu nhất cho model **lắp lại nhiều lần**, ít **quyết định** (Critic); giữ độ chính xác cao nhất cho model **chạy 1 lần**, **ảnh hưởng toàn cục** (Planner).

Ngoại lệ: Không Quantize Embedding & Reranker

Đầu ra của chúng là **toạ độ/điểm số liên tục dùng để so sánh** (similarity, ranking) - nhiều lượng tử hoá làm lệch khoảng cách giữa các ứng viên gần nhau, trong khi LLM chỉ chọn 1 token qua argmax nên chịu nhiễu tốt hơn nhiễu; lỗi này cũng **không có Critic nào bắt lại**.

Lin et al., 2024 (MLSys) - "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration" - arxiv.org/abs/2306.00978

Một Base Model, Nhiều Vai trò: Multi-LoRA Serving

Thay vì host N model riêng biệt cho N vai trò, dùng **1 base model dùng chung** + nhiều adapter LoRA nhỏ (một adapter/vai trò) - vLLM phục vụ đồng thời hàng trăm adapter trên cùng GPU vì phần trọng số nền tảng (chiếm phần lớn VRAM) chỉ tồn tại đúng 1 bản.

Base Model (dùng chung, load 1 lần)



+ Adapter Planner + Adapter Retriever
+ Adapter Programmer + Adapter Critic

Sheng et al., 2023 - "*S-LoRA: Serving Thousands of Concurrent LoRA Adapters*" - arxiv.org/abs/2311.03285

Latency Budget cho Pipeline Multi-Agent

Độ trễ Cộng dồn qua Chuỗi Agent Tuần tự

Chạy tuần tự Planner → Retriever → Programmer → Sandbox → Critic - mỗi bước tốn từ vài trăm ms đến vài giây, **độ trễ cộng dồn theo tổng**, không theo bước chậm nhất.

Planner → Retriever → Programmer → Sandbox → Critic

↓
Tổng độ trễ = $\sum$ từng bước - có thể lên tới hàng chục giây/câu hỏi

Vì sao nguy hiểm

Khác với lỗi tính toán sai (có thể phát hiện bằng verification), độ trễ tích lũy **không có cơ chế tự báo lỗi** - hệ thống vẫn "đúng" nhưng không dùng được trong sản phẩm tương tác thời gian thực.

Song song hoá Các Bước Thực sự Độc lập

Không phải mọi bước đều phụ thuộc tuần tự - cần tách rõ **phụ thuộc dữ liệu thật** khỏi **thói quen viết pipeline tuần tự**.

Song song hoá được

Nhiều Critic kiểm tra các khía cạnh độc lập cùng lúc (logic kế toán, đối chiếu số liệu, format câu trả lời); Retriever định vị nhiều bảng không phụ thuộc nhau trong cùng 1 câu hỏi multi-hop.

Buộc phải tuần tự

Programmer cần chờ đúng bảng Retriever trả về; Sandbox cần chờ đúng code Programmer sinh ra - phụ thuộc dữ liệu thật, không thể song song.

Sandbox Thực thi An toàn

Không Bao giờ Tin Code do LLM Sinh ra

- ▶ Code Pandas sinh ra được **thực thi trực tiếp trên server thật** - sai sót ở đây không chỉ là kết quả sai, mà có thể là **mất quyền kiểm soát hệ thống**.
- ▶ Dữ liệu vào hệ thống có nguồn OCR từ bên thứ ba → có thể chứa chỉ thị ẩn dụ LLM sinh code độc (*indirect prompt injection*).
- ▶ Cho agent quyền "viết code rồi tự chạy" = cấp một năng lực lớn (**Excessive Agency** - theo phân loại của OWASP, tổ chức phi lợi nhuận chuyên về bảo mật ứng dụng).

Nguyên tắc

Code do LLM sinh ra **không bao giờ được coi là an toàn**. Giải pháp không phải "kiểm tra cho sạch" mà là chạy trong môi trường **dù độc hại thật cũng không gây hại được**.

"OWASP Top 10 for LLM Applications 2025" -

genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025

Sandbox có Hai Tầng Độc lập

Tầng	Đặc điểm
Ngôn ngữ	Giới hạn cú pháp/thư viện - vẫn chạy chung tiến trình với ứng dụng chủ
Hệ điều hành	Cách ly hẳn tiến trình/kernel - ranh giới cứng, an toàn bất kể code "sạch" hay không

Hai tầng không thay thế nhau

Tầng ngôn ngữ chỉ nên là lớp lọc sơ bộ, rẻ/nhanh. Tầng hệ điều hành mới là ranh giới an toàn thật, chịu trách nhiệm chính.

Tầng Ngôn ngữ: Vì sao Không Đủ Tin cậy

Ví dụ: **RestrictedPython** cấm cú pháp nguy hiểm - nhưng code vẫn chạy **chung bộ nhớ** với app chủ, nên luôn có "đường vòng nội bộ" chưa lường hết:

```
pandas → numpy → numpy.ma.core → inspect → sys → sys.modules → os →  
os.system(...)
```

Chỉ cần **một** thư viện cho phép, như pandas/numpy, đã kéo theo cả chuỗi phụ thuộc - một đường vòng còn sót là đủ để thoát sandbox.

Tầng Hệ điều hành: Các Phương pháp Cách ly

#	Phương pháp	Cơ chế	Mức cách ly
1	Chroot + user riêng	Đổi gốc filesystem + user hạn chế	Yếu
2	Container (Docker)	Namespace + cgroups	Trung bình
3	gVisor	Chặn syscall, kernel giả lập user-space	Cao
4	MicroVM (Fire-cracker)	Ảo hoá phần cứng, kernel riêng	Cao nhất

Thank you for your attention!

Tài liệu Tham khảo (References) I

- [1] Yilun Zhao, Yunxiang Li, Chenying Li, and Rui Zhang. MultiHiertt: Numerical reasoning over multi hierarchical tabular and textual data. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 6588–6600, 2022. doi: 10.18653/v1/2022.acl-long.454. URL <https://aclanthology.org/2022.acl-long.454/>.
- [2] Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. HiTab: A hierarchical table dataset for question answering and natural language generation. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1094–1110, 2022. doi: 10.18653/v1/2022.acl-long.78. URL <https://aclanthology.org/2022.acl-long.78/>.
- [3] Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. TaPas: Weakly supervised table parsing via pre-training. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 4320–4333, 2020. doi: 10.18653/v1/2020.acl-main.398. URL <https://aclanthology.org/2020.acl-main.398/>.
- [4] Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), pages 3277–3287, 2021. doi: 10.18653/v1/2021.acl-long.254. URL <https://aclanthology.org/2021.acl-long.254/>.
- [5] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. FinQA: A dataset of numerical reasoning over financial data. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pages 3697–3711, 2021. doi: 10.18653/v1/2021.emnlp-main.300. URL <https://aclanthology.org/2021.emnlp-main.300/>.
- [6] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. Transactions on Machine Learning Research, 2023. URL <https://openreview.net/forum?id=YfZ4ZPt8zd>.

Tài liệu Tham khảo (References) II

- [7] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In Proceedings of the 40th International Conference on Machine Learning, volume 202 of Proceedings of Machine Learning Research, pages 10764–10799, 2023. URL <https://proceedings.mlr.press/v202/gao23f.html>.
- [8] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. Binding language models in symbolic languages. In The Eleventh International Conference on Learning Representations, 2023. URL <https://openreview.net/forum?id=1H1PV42cbF>.
- [9] Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In The Twelfth International Conference on Learning Representations, 2024. URL <https://openreview.net/forum?id=4L0xnS4GQM>.
- [10] Si-An Chen, Lesly Miculicich, Julian Martin Eisenschlos, Zifeng Wang, Zilong Wang, Yanfei Chen, Yasuhisa Fujii, Hsuan-Tien Lin, Chen-Yu Lee, and Tomas Pfister. TableRAG: Million-token table understanding with language models. In Advances in Neural Information Processing Systems, volume 37, 2024. doi: 10.52202/079017-2382. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/88dd7aa6979e352fda7c4952ca8eac59-Abstract-Conference.html.
- [11] Shuangyan Deng, Zhongsheng Wang, Rui Mao, Ciprian Doru Giurcaneanu, and Jiamou Liu. CLER: Improving multimodal financial reasoning by cross-MLLM error reflection. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 40, pages 30494–30502, 2026. doi: 10.1609/aaai.v40i36.40303. URL <https://ojs.aaai.org/index.php/AAAI/article/view/40303>.